

Bayesian mixture models

José G. Dias

Table of contents

1. Summary

This lecture introduces mixture modeling. It also illustrates the problem of lacking identifiability in the context of MCMC.

2. Gaussian mixture models

In the previous lectures, we observed a lack of symmetry in the distribution of mpg, which may suggest a mixture model.

```
library(tidyverse)
```

```
— Attaching core tidyverse packages ————— tidyverse 2.0.0
```

```
✓ dplyr      1.1.4    ✓ readr      2.1.5
✓ forcats    1.0.0    ✓ stringr    1.5.1
✓ ggplot2    3.5.1    ✓ tibble     3.2.1
✓ lubridate  1.9.3    ✓ tidyr      1.3.1
✓ purrr      1.0.2
```

```
— Conflicts ————— tidyverse_conflicts() —
```

```
✖ dplyr::filter() masks stats::filter()
```

```
✖ dplyr::lag() masks stats::lag()
```

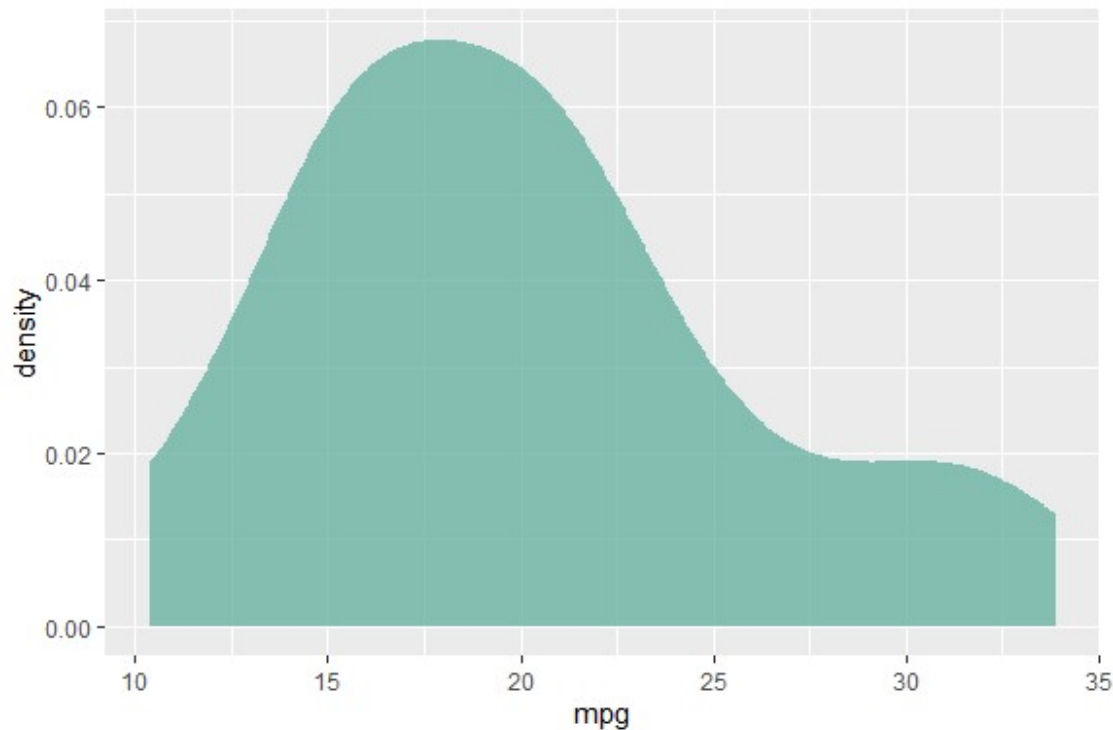
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

```
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
set.seed(1234)
```

```
ggplot(mtcars, aes(x=mpg)) +  
  geom_density(fill="#69b3a2", color="#e9ecef", alpha=0.8)
```



This graph suggests two clusters of cars. Then, we can assume a Gaussian mixture model (GMM) with two components:

$$f(y; w, \mu_1, \mu_2, \sigma_1^2, \sigma_2^2) = wN(y; \mu_1, \sigma_1^2) + (1 - w)N(y; \mu_2, \sigma_2^2).$$

The prior distributions for our parameters (weak priors) are:

$$\mu_1 \sim N(0, 10000)$$

$$\mu_2 \sim N(0, 10000)$$

$$\sigma_1 \sim \text{Half-Cauchy}(0, 2.5)$$

$$\sigma_2 \sim \text{Half-Cauchy}(0, 2.5)$$

For the parameter w , continuous in $[0, 1]$, we can set a beta prior as we did before for the probability of success. However, in the case of more than two clusters, e.g. 3, we have $(w_1, w_2, 1 - w_1 - w_2)$. In this case, we use the Dirichlet distribution that works for any number of components/clusters (compositional data).

3. GMM in Stan

```
#install.packages('rstan')  
library(rstan)
```

Loading required package: StanHeaders

rstan version 2.32.6 (Stan version 2.32.2)

For execution on a local, multicore CPU with excess RAM we recommend calling `options(mc.cores = parallel::detectCores())`.

To avoid recompilation of unchanged Stan programs, we recommend calling `rstan_options(auto_write = TRUE)`

For within-chain threading using ``reduce_sum()`` or ``map_rect()`` Stan functions,

change ``threads_per_chain`` option:

`rstan_options(threads_per_chain = 1)`

Do not specify `'-march=native'` in `'LOCAL_CPPFLAGS'` or a Makevars file

Attaching package: 'rstan'

The following object is masked from 'package:tidyr':

`extract`

`library(posterior)`

This is posterior version 1.5.0

Attaching package: 'posterior'

The following objects are masked from 'package:rstan':

`ess_bulk, ess_tail`

The following objects are masked from 'package:stats':

`mad, sd, var`

The following objects are masked from 'package:base':

`%in%, match`

`library(bayesplot)`

This is bayesplot version 1.11.1

- Online documentation and vignettes at mc-stan.org/bayesplot

- bayesplot theme set to `bayesplot::theme_default()`

* Does `_not_` affect other ggplot2 plots

* See `?bayesplot_theme_set` for details on theme setting

Attaching package: 'bayesplot'

The following object is masked from 'package:posterior':

rhat

library(coda)

Attaching package: 'coda'

The following object is masked from 'package:rstan':

traceplot

Define the GMM in Stan:

Stan model code

```
stan_model.GMM <- '
data {
  int<lower=1> N; // number of observations
  real y[N];     // response variable - normally distributed
  int<lower=2> K; // number of mixture distributions
}

parameters {
  ordered[K] mu;           // enforcing an ordering
  real<lower=0> sigma[K];  //
  simplex[K] pis;         // weights (should sum to one)
}

model {
  real ps[K]; // temporary variable for log components

  // Priors
  mu[1] ~ normal(0, 10000);
  mu[2] ~ normal(0, 10000);
  sigma[1] ~ cauchy(0, 2.5); // weak prior
  sigma[2] ~ cauchy(0, 2.5); // weak prior
  pis ~ dirichlet(rep_vector(3.0, K));

  // loop to calculate likelihood
  for(n in 1:N){
    // second loop for number of mixture components
    for (k in 1:K){
      ps[k] = log(pis[k]) + normal_lpdf(y[n] | mu[k], sigma[k]);
    }
    target += log_sum_exp(ps);
  }
}
```

```
}  
}'
```

```
# Running stan code
```

```
model.GMM <- stan_model(model_code = stan_model.GMM)
```

```
# data set
```

```
data <- list(N = length(mtcars$mpg), y = mtcars$mpg, K = 2)
```

```
# Run the Stan model
```

```
stan_fit.GMM <- sampling(model.GMM, data = data, chains = 4,  
                          iter = 4000, warmup = 1000)
```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 8.9e-05 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.89 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 4000 [0%] (Warmup)

Chain 1: Iteration: 400 / 4000 [10%] (Warmup)

Chain 1: Iteration: 800 / 4000 [20%] (Warmup)

Chain 1: Iteration: 1001 / 4000 [25%] (Sampling)

Chain 1: Iteration: 1400 / 4000 [35%] (Sampling)

Chain 1: Iteration: 1800 / 4000 [45%] (Sampling)

Chain 1: Iteration: 2200 / 4000 [55%] (Sampling)

Chain 1: Iteration: 2600 / 4000 [65%] (Sampling)

Chain 1: Iteration: 3000 / 4000 [75%] (Sampling)

Chain 1: Iteration: 3400 / 4000 [85%] (Sampling)

Chain 1: Iteration: 3800 / 4000 [95%] (Sampling)

Chain 1: Iteration: 4000 / 4000 [100%] (Sampling)

Chain 1:

Chain 1: Elapsed Time: 0.665 seconds (Warm-up)

Chain 1: 1.888 seconds (Sampling)

Chain 1: 2.553 seconds (Total)

Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 3.4e-05 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.34 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 4000 [0%] (Warmup)

Chain 2: Iteration: 400 / 4000 [10%] (Warmup)
Chain 2: Iteration: 800 / 4000 [20%] (Warmup)
Chain 2: Iteration: 1001 / 4000 [25%] (Sampling)
Chain 2: Iteration: 1400 / 4000 [35%] (Sampling)
Chain 2: Iteration: 1800 / 4000 [45%] (Sampling)
Chain 2: Iteration: 2200 / 4000 [55%] (Sampling)
Chain 2: Iteration: 2600 / 4000 [65%] (Sampling)
Chain 2: Iteration: 3000 / 4000 [75%] (Sampling)
Chain 2: Iteration: 3400 / 4000 [85%] (Sampling)
Chain 2: Iteration: 3800 / 4000 [95%] (Sampling)
Chain 2: Iteration: 4000 / 4000 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 0.638 seconds (Warm-up)
Chain 2: 5.771 seconds (Sampling)
Chain 2: 6.409 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

Chain 3:
Chain 3: Gradient evaluation took 9.2e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.92 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration: 1 / 4000 [0%] (Warmup)
Chain 3: Iteration: 400 / 4000 [10%] (Warmup)
Chain 3: Iteration: 800 / 4000 [20%] (Warmup)
Chain 3: Iteration: 1001 / 4000 [25%] (Sampling)
Chain 3: Iteration: 1400 / 4000 [35%] (Sampling)
Chain 3: Iteration: 1800 / 4000 [45%] (Sampling)
Chain 3: Iteration: 2200 / 4000 [55%] (Sampling)
Chain 3: Iteration: 2600 / 4000 [65%] (Sampling)
Chain 3: Iteration: 3000 / 4000 [75%] (Sampling)
Chain 3: Iteration: 3400 / 4000 [85%] (Sampling)
Chain 3: Iteration: 3800 / 4000 [95%] (Sampling)
Chain 3: Iteration: 4000 / 4000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 0.626 seconds (Warm-up)
Chain 3: 1.144 seconds (Sampling)
Chain 3: 1.77 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).

Chain 4:
Chain 4: Gradient evaluation took 3.6e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.36 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:


```

mu[2]      2568.91 1324.48 4974.61   19.59   23.33   29.11 2844.67 17336.73
14
sigma[1]    4.55    0.41    1.63    1.28    3.41    4.44    5.75    7.58
15
sigma[2]   10.80    1.98   82.87    0.34    2.58    5.04    6.72   27.00
1747
pis[1]      0.69    0.06    0.23    0.19    0.53    0.75    0.90    0.97
13
pis[2]      0.31    0.06    0.23    0.03    0.10    0.25    0.47    0.81
13
lp__       -105.75    0.25    2.10 -110.63 -106.97 -105.44 -104.18 -102.67
71

      Rhat
mu[1]   1.14
mu[2]   1.18
sigma[1] 1.14
sigma[2] 1.00
pis[1]   1.15
pis[2]   1.15
lp__     1.05

```

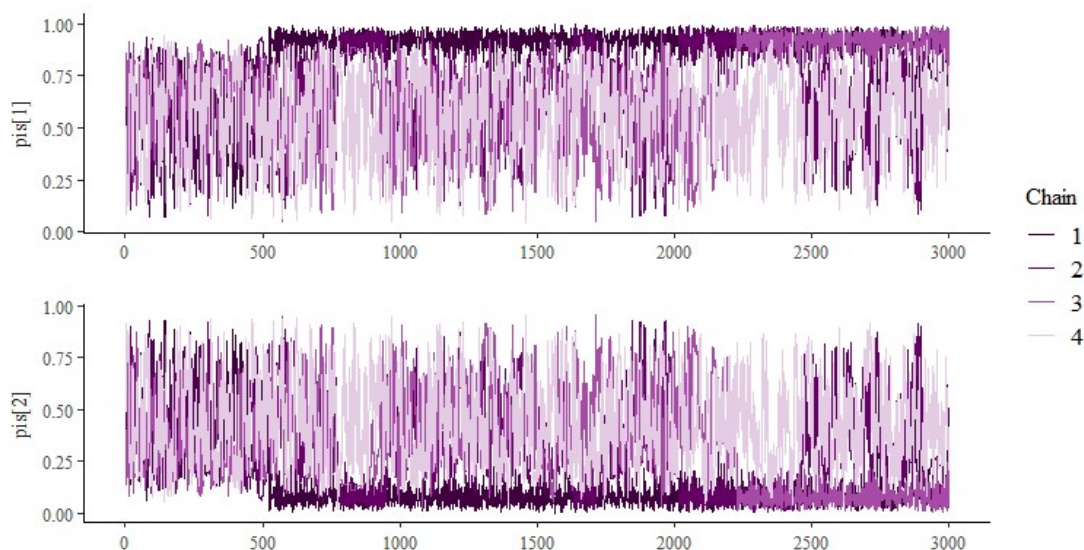
Samples were drawn using NUTS(diag_e) at Thu Mar 13 12:56:39 2025.
 For each parameter, `n_eff` is a crude measure of effective sample size,
 and `Rhat` is the potential scale reduction factor on split chains (at
 convergence, `Rhat=1`).

```
post.GMM <- as.array(stan_fit.GMM)
```

```

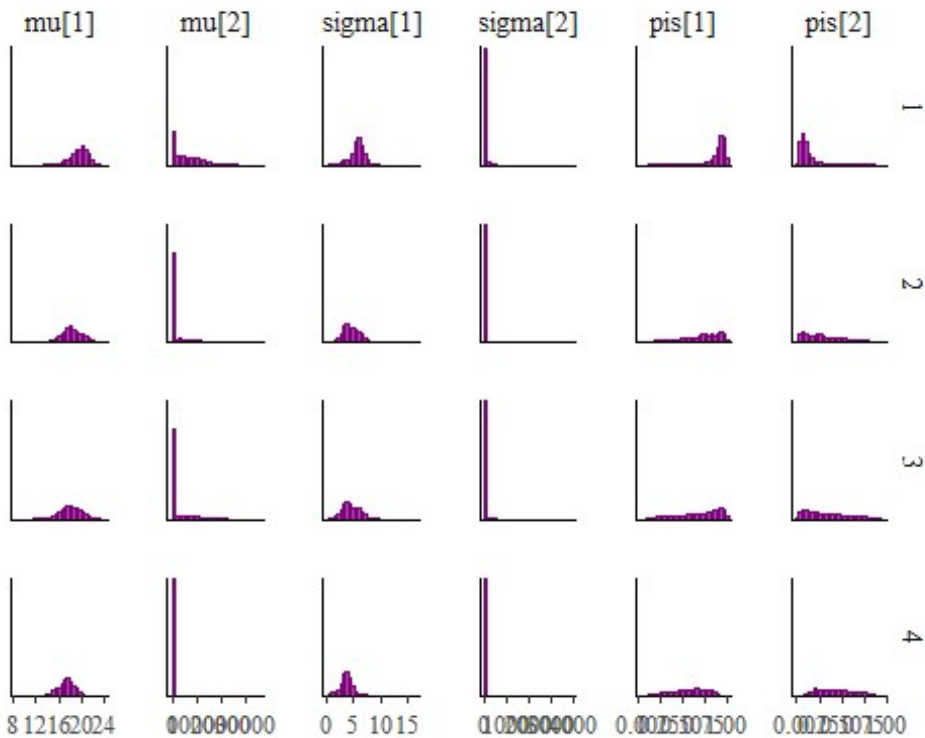
#Quality
color_scheme_set("purple")
mcmc_trace(post.GMM, pars = c("pis[1]", "pis[2]"),
            facet_args = list(ncol = 1, strip.position = "left"))

```

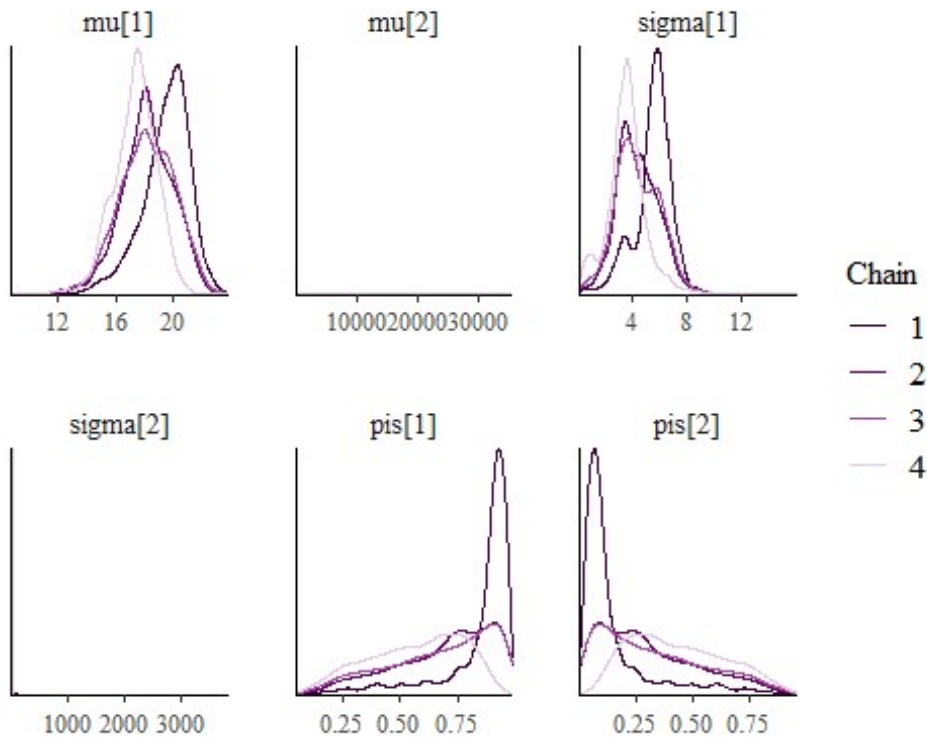



```
mcmc_hist_by_chain(post.GMM, pars = c("mu[1]", "mu[2]",
"sigma[1]", "sigma[2]", "pis[1]", "pis[2]"))

`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



```
mcmc_dens_overlay(post.GMM, pars = c("mu[1]", "mu[2]",
"sigma[1]", "sigma[2]", "pis[1]", "pis[2]"))
```



```
# Analyze results - Bayesian statistics
```

```
# Extract parameter names
```

```
param_names <- colnames(post.GMM)
```

```
# Compute mean and sd for each parameter
```

```
means <- colMeans(post.GMM)
```

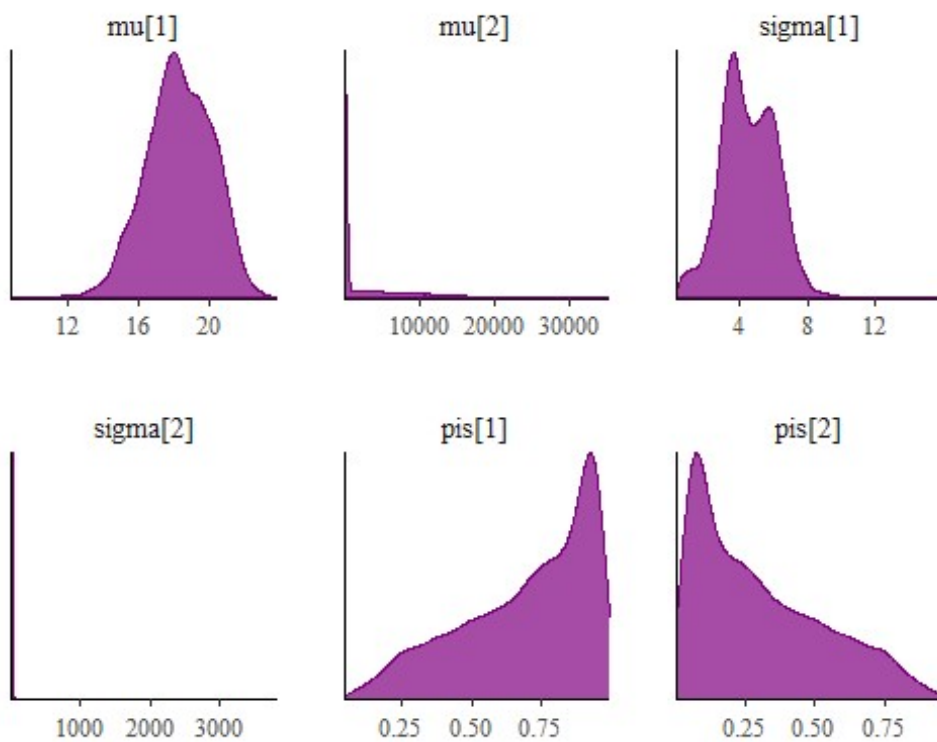
```
# Display the results
```

```
param_statistics <- data.frame(Parameter = param_names, Mean = means[,1:5])
print(param_statistics)
```

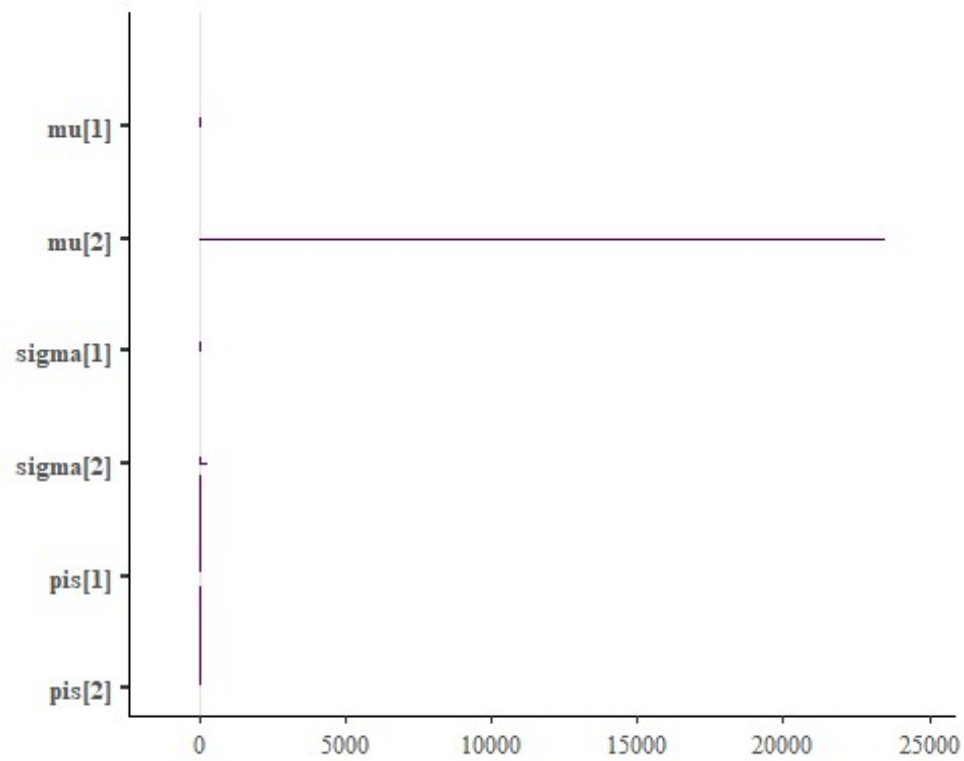
	Parameter	Mean.mu.1.	Mean.mu.2.	Mean.sigma.1.	Mean.sigma.2.
Mean.pis.1.					
chain:1	chain:1	19.46164	6023.77191	5.559586	22.368643
0.8439146					
chain:2	chain:2	18.18647	2108.63589	4.476203	8.226075
0.6923961					
chain:3	chain:3	18.15680	2117.77648	4.479707	7.061736
0.6738891					
chain:4	chain:4	17.30792	25.45002	3.688217	5.538512
0.5667410					

```
# Density plot
```

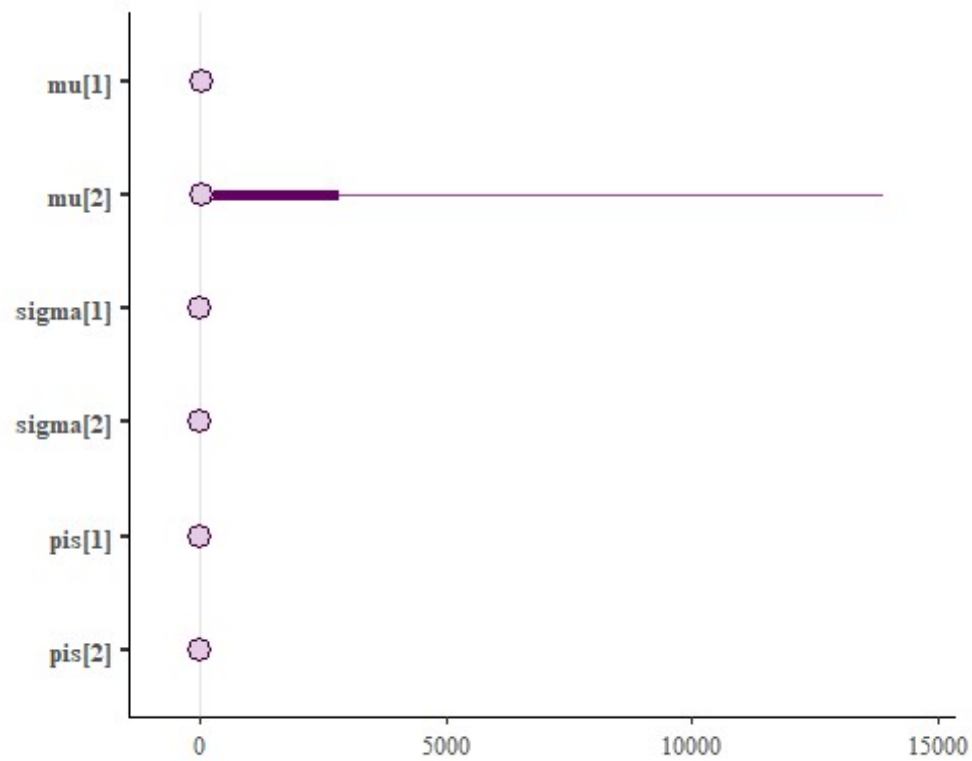
```
mcmc_dens(post.GMM, pars = c("mu[1]", "mu[2]",  
"sigma[1]", "sigma[2]", "pis[1]", "pis[2]"))
```



```
mcmc_areas(
  post.GMM,
  pars = c("mu[1]", "mu[2]", "sigma[1]", "sigma[2]", "pis[1]", "pis[2]"),
  prob = 0.8, # 80% intervals
  prob_outer = 0.99, # 99%
  point_est = "mean"
)
```



```
mcmc_intervals(post.GMM, pars = c("mu[1]", "mu[2]",  
"sigma[1]", "sigma[2]", "pis[1]", "pis[2]"))
```



```
# Pairs plot
```

```
mcmc_pairs(post.GMM, pars = c("mu[1]", "mu[2]",  
"sigma[1]", "sigma[2]", "pis[1]", "pis[2]"))
```

